

Versi 2.0
September 2026



PEMROGRAMAN WEB

MODUL 3 - PHP PROSEDURAL & PHP OOP

Disusun Oleh:

Christian Sri Kusuma Aditya, S.Kom., M.Kom

Shofwan Alif Alfani

Nisrina Nurhafizhah

LABORATORIUM INFORMATIKA
Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	3
TUUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KEYWORDS.....	3
MATERI.....	4
1. PHP dalam pengembangan aplikasi web.....	4
1.1. Sintaks Dasar PHP.....	4
• Variabel dan Tipe Data.....	5
• Kondisional.....	6
• Looping.....	7
• Fungsi.....	7
• Array dan fungsi bawaannya.....	8
1.2. PHP prosedural dan keterbatasannya.....	9
2. Object-Oriented Programming (OOP) menggunakan PHP.....	10
2.1. Mengapa membutuhkan OOP.....	10
2.2. Class dan object.....	11
2.3. Property dan method.....	12
2.4. Constructor.....	13
2.5. Encapsulation dan Access Modifier.....	14
2.6. Inheritance.....	16
3. Mengorganisir kode pada PHP.....	18
3.1. Struktur Project.....	18
3.2. Namespace.....	19
3.3. Type Hinting dan Return Type.....	20
4. Perancangan backend engine sederhana.....	21
4.1. Identifikasi entitas sistem.....	21
4.2. Implementasi prototype backend engine.....	22
CODELAB.....	27
DEMO.....	28
Ketentuan Demo.....	28
Simulasi Yang Diharapkan.....	28
RUBRIK PENILAIAN.....	29
SEDIKIT CATATAN.....	30



PENDAHULUAN

TUUAN

1. Memahami sintaks dasar PHP untuk pembuatan web
2. Memahami konsep Object-Oriented Programming (OOP) pada PHP
3. Memahami penerapan OOP dalam perancangan struktur backend sederhana

TARGET MODUL

1. Mahasiswa mampu menulis kode PHP menggunakan sintaks yang sesuai dan terstruktur
2. Mahasiswa mampu mengimplementasikan konsep OOP menggunakan PHP
3. Mahasiswa mampu merancang prototipe backend engine berdasarkan studi kasus tertentu menggunakan PHP

PERSIAPAN

1. Bagi mahasiswa Muslim, disunnahkan membaca doa berikut sebelum memulai praktikum
رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop atau komputer & koneksi internet
3. PHP versi 8.x atau versi terbaru yang tersedia
4. Text editor (VS Code atau sejenisnya)
5. VS Code extension PHP Intelephense, Code Runner
6. Motivasi dan komitmen belajar yang kuat

KEYWORDS

PHP, Object Oriented Programming (OOP), Backend



MATERI

1. PHP dalam pengembangan aplikasi web

PHP adalah bahasa pemrograman server-side yang dirancang untuk pengembangan web. Jika di modul sebelumnya kita mengetahui JavaScript adalah bahasa pemrograman yang berjalan di sisi browser (client-side), maka sebaliknya, PHP dieksekusi di server dan hasilnya dikirim ke client. Dalam arsitektur web modern, PHP umumnya bertugas sebagai:

- Memproses data dari form atau request yang dikirim dari client
- Berkomunikasi atau berinteraksi dengan database (baca, tulis, ubah, hapus data)
- Menerapkan logika bisnis pada sebuah aplikasi web

1.1. Sintaks Dasar PHP

Setiap file PHP selalu diawali dengan tag pembuka `<?php` dan diakhiri dengan tag penutup `?>`. Contoh:

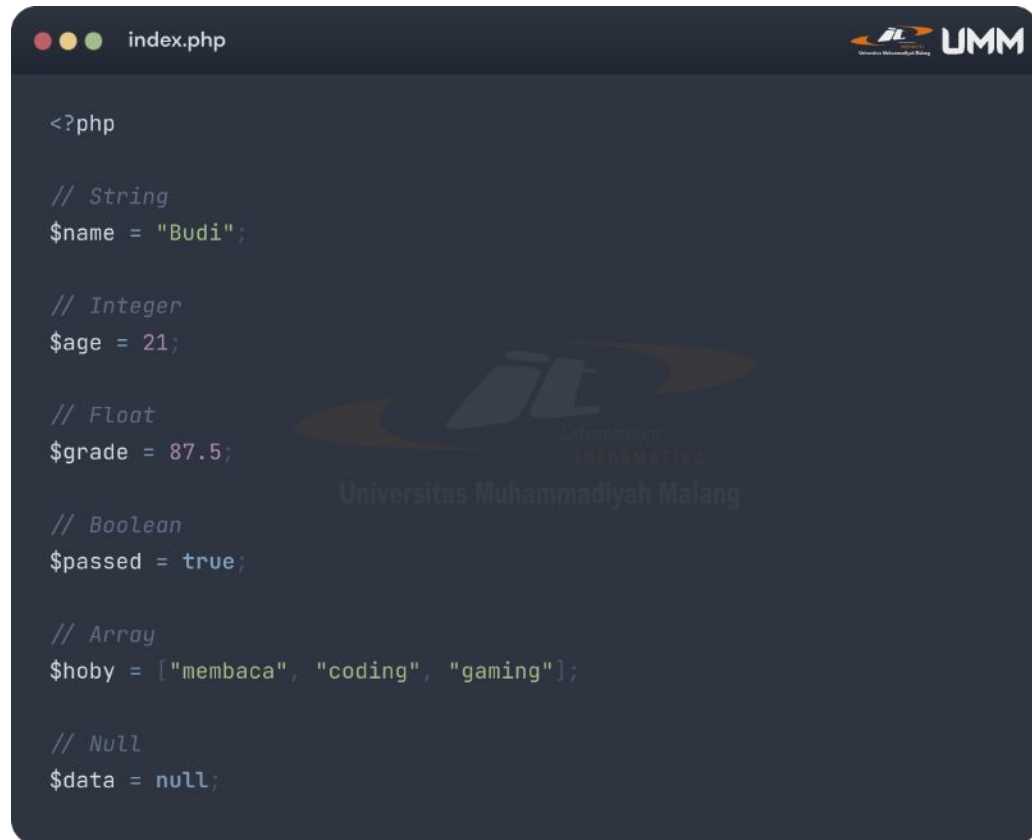


Gambar 1.1: Sintaks dasar PHP



- **Variabel dan Tipe Data**

Setiap deklarasi variabel di PHP selalu diawali dengan tanda \$ dan tidak perlu dideklarasikan tipenya secara eksplisit. Contoh:



```

<?php

// String
$name = "Budi";

// Integer
$age = 21;

// Float
$grade = 87.5;

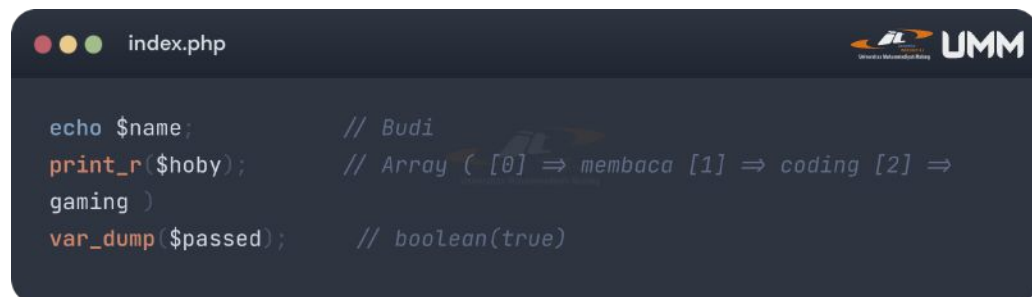
// Boolean
$passed = true;

// Array
$hoby = ["membaca", "coding", "gaming"];

// Null
$data = null;
  
```

Gambar 1.2: Variabel dan tipe data PHP

Kemudian, untuk cara memanggil atau menampilkan nilai dari variabel bisa dengan cara berikut:



```

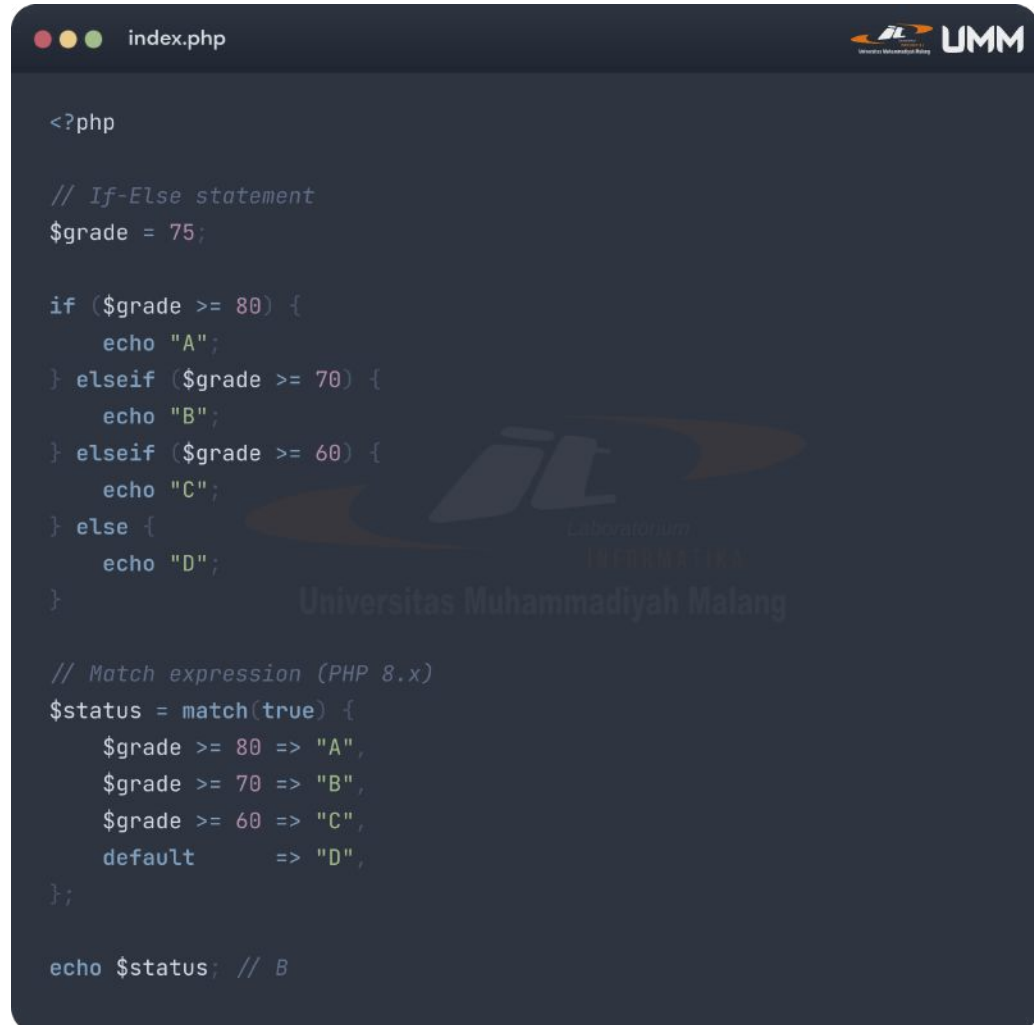
echo $name;           // Budi
print_r($hoby);       // Array ( [0] => membaca [1] => coding [2] => gaming )
var_dump($passed);    // boolean(true)
  
```

Gambar 1.3: Menampilkan nilai dari variabel



- **Kondisional**

Kondisional memungkinkan program memilih tindakan yang berbeda berdasarkan kondisi tertentu. Contoh:



```

index.php

<?php

// If-Else statement
$grade = 75;

if ($grade >= 80) {
    echo "A";
} elseif ($grade >= 70) {
    echo "B";
} elseif ($grade >= 60) {
    echo "C";
} else {
    echo "D";
}

// Match expression (PHP 8.x)
$status = match(true) {
    $grade >= 80 => "A",
    $grade >= 70 => "B",
    $grade >= 60 => "C",
    default      => "D",
};

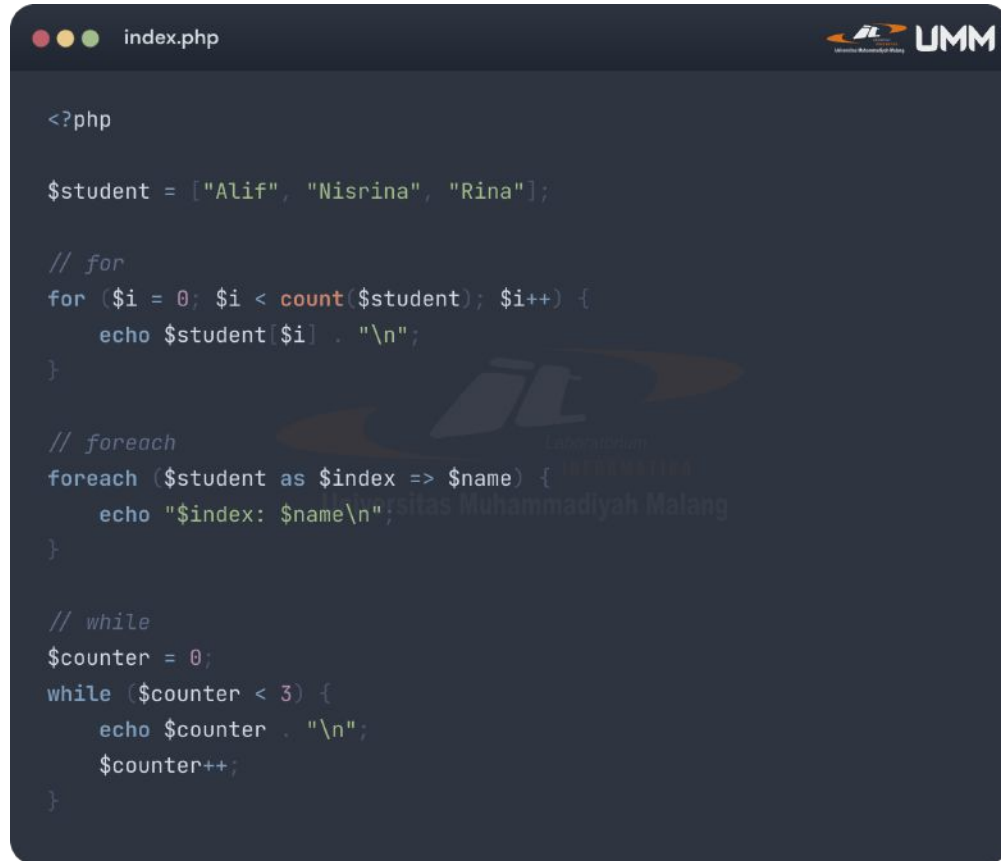
echo $status; // B
    
```

Gambar 1.4: Kondisional pada PHP



• Looping

Perulangan (looping) digunakan untuk menjalankan blok kode secara berulang selama kondisi tertentu masih terpenuhi atau sampai seluruh data diproses. Jenis perulangan yang sering digunakan adalah **for**, **foreach**, **while**



```
<?php

$student = ["Alif", "Nisrina", "Rina"];

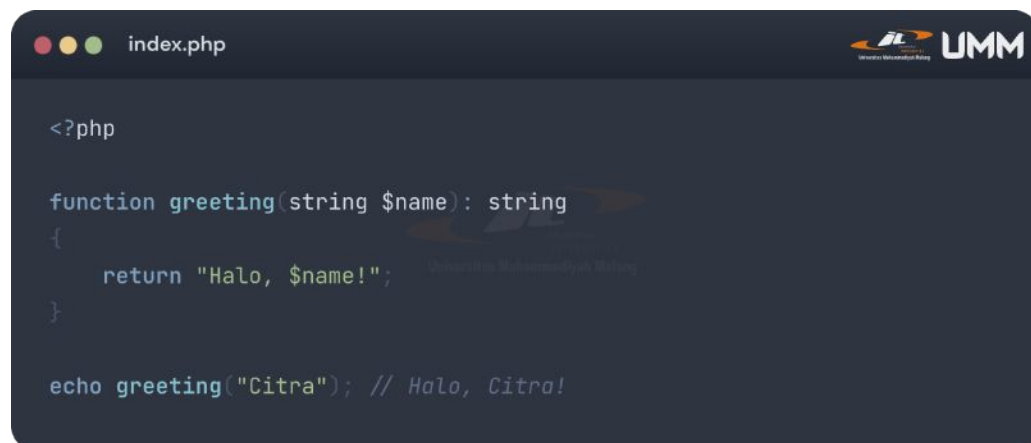
// for
for ($i = 0; $i < count($student); $i++) {
    echo $student[$i] . "\n";
}

// foreach
foreach ($student as $index => $name) {
    echo "$index: $name\n";
}

// while
$counter = 0;
while ($counter < 3) {
    echo $counter . "\n";
    $counter++;
}
```

Gambar 1.5: Looping PHP

• Fungsi



```
<?php

function greeting(string $name): string
{
    return "Halo, $name!";
}

echo greeting("Citra"); // Halo, Citra!
```

Gambar 1.6: Deklarasi fungsi PHP



- **Array dan fungsi bawaannya**

Array digunakan untuk menyimpan banyak nilai dalam satu variabel



```

index.php

<?php

$number = [5, 3, 8, 1, 9, 2];

// Sorting
sort($number);
print_r($number); // [1, 2, 3, 5, 8, 9]

// Filter
$bigger = array_filter($number, fn($n) => $n > 4);
print_r($bigger); // [5, 8, 9]

// Map
$timesTwo = array_map(fn($n) => $n * 2, $number);
print_r($timesTwo); // [2, 4, 6, 10, 16, 18]
  
```

Gambar 1.7: Array dan fungsi bawaannya pada PHP

Catatan: masih banyak fungsi array lain yang bisa kalian coba, seperti `count()`, `array_push()`, `array_pop()`, dan lainnya.



1.2. PHP prosedural dan keterbatasannya

Sebelum masuk ke PHP OOP, penting untuk memahami cara kerja PHP prosedural dan mengapa pendekatan prosedural mulai tidak memadai seiring meningkatnya kompleksitas aplikasi web yang kita buat. Jika menggunakan pendekatan prosedural, kode ditulis sebagai serangkaian fungsi dan data yang berdiri sendiri. Contoh:



```
<?php

function getStudentData(string $nim): array
{
    // Simulasi ambil data
    return [
        "nim" => $nim,
        "name" => "Shofwan Alif",
        "major" => "Informatics Engineering",
    ];
}

function getStudentGrades(string $nim): array
{
    return [
        "nim" => $nim,
        "course" => "Pemrograman Web",
        "final_grade" => 85,
    ];
}

function printReport(string $nim): void
{
    $biodata = getStudentData($nim);
    $nilai = getStudentGrades($nim);

    echo "NIM : {$biodata['nim']}\n";
    echo "Name : {$biodata['name']}\n";
    echo "Final Grade : {$nilai['final_grade']}\n";
}

printReport("202310370311238");
```

Gambar 1.8: Contoh kode PHP prosedural



Kode tersebut masih terlihat bersih karena skalanya masih kecil. Fungsi `getStudentData()`, `getStudentGrades()`, dan `printReport()` berdiri sendiri untuk memproses data yang diinput. Masalah mulai muncul ketika fitur bertambah; ratusan fungsi baru akan dibuat dan tersebar di berbagai file. Tanpa adanya pengelompokan yang jelas, kita akan mulai kesulitan melacak fungsi mana yang sebenarnya milik entitas tertentu. Semua fungsi bercampur sehingga meningkatkan risiko terjadinya bentrokan nama fungsi (*name collision*) dan membuat alur program sulit dipetakan serta sulit di-maintenance. Di sinilah *Object-Oriented Programming* (OOP) menjadi sangat relevan.

2. Object-Oriented Programming (OOP) menggunakan PHP

2.1. Mengapa membutuhkan OOP

Jika pada pendekatan prosedural kita memperlakukan kode sebagai rangkaian instruksi yang terpisah dari datanya, OOP menawarkan cara pandang yang berbeda. OOP adalah paradigma pemrograman yang mengorganisasi seluruh arsitektur kode berdasarkan objek. Objek yang dimaksud merupakan representasi dari entitas di dunia nyata maupun konsep abstrak yang memiliki dua komponen utama, yaitu **data** (yang disebut sebagai *property*) dan **perilaku** (yang dikenal sebagai *method*). Dengan pendekatan ini, kode yang sebelumnya tersebar acak di berbagai file kini disatukan menjadi satu unit yang terstruktur. Ketika membangun aplikasi web modern, penerapan OOP memberikan fondasi yang kuat melalui empat pilar utamanya yang disesuaikan untuk kebutuhan web:

Tabel 2.1: Konsep OOP

Konsep	Manfaat
Encapsulation	Membungkus data (<i>property</i>) dan perilaku (<i>method</i>) ke dalam satu kelas, serta membatasi akses langsung dari luar
Inheritance	Kemampuan sebuah kelas untuk menurunkan atau mewariskan property dan methodnya kepada kelas lain
Polymorphism	Kemampuan beberapa objek berbeda untuk merespons instruksi atau antarmuka (<i>interface</i>) yang sama dengan cara mereka masing-masing
Abstraction	Menyembunyikan kompleksitas detail implementasi dan hanya menampilkan fungsi esensialnya saja



2.2. Class dan object

Class adalah cetak biru (blueprint), sedangkan object adalah instance yang dibuat dari class tersebut. Jika dianalogikan seperti membuat kue, class adalah cetakan kue, sedangkan object adalah kue yang sudah jadi berdasarkan cetakan tersebut. Dari satu cetakan itu, bisa dibuat kue dengan karakteristik yang sama, namun nilainya berbeda-beda. Contoh:



```

index.php

<?php

class Students
{
    //class contents are usually methods and properties
}

$student1 = new Mahasiswa();
$student2 = new Mahasiswa();

// $student1 and $student2 are two different objects
// both are instances of the Students class
    
```

Gambar 2.1: class dan object PHP OOP

`$student1` dan `$student2` adalah dua objek yang berbeda, tetapi keduanya sama-sama merupakan instance dari class `Students`.



2.3. Property dan method

Property adalah variabel yang melekat pada sebuah class. Sedangkan method adalah fungsi yang melekat pada sebuah class. Untuk mengakses properti dan metode dari sebuah objek, PHP menggunakan dua operator khusus yang akan terus muncul dalam kode OOP.

- `->` (object operator) digunakan untuk mengakses property atau method dari sebuah object. Di dalam class, object yang sedang aktif dirujuk menggunakan `$this`, sehingga untuk mengakses property milik sendiri ditulis `$this->namaProperty`
- `::` (scope resolution operator) – digunakan untuk mengakses sesuatu yang melekat pada class itu sendiri, bukan pada instance tertentu. Konteks penggunaannya akan muncul secara bertahap: untuk konstanta class via `self::`, untuk memanggil method parent via `parent::`, dan untuk static method via `NamaClass::`

Contohnya seperti ini:



```
<?php

class Students
{
    // Property
    public string $nim;
    public string $name;
    public string $major;

    // Method
    public function introduction(): string
    {
        return "i'am {$this->nama}, NIM {$this->nim} from
{$this->major}.";
    }

    public function getFinalGrade(int $uts, int $uas): float
    {
        return ($uts * 0.4) + ($uas * 0.6);
    }
}

// create objects and fill in properties
$mhs = new Mahasiswa();
$mhs->nim    = "202310370311238";
$mhs->name  = "Shofwan Alif";
$mhs->major = "Teknik Informatika";

// call method
echo $mhs->introduction();
// Output: i'am Shofwan Alif, NIM 202310370311238 from Teknik
Informatika.

echo $mhs->getFinalGrade(80, 90);
// Output: 86
```

Gambar 2.2: Property dan method PHP OOP



2.4. Constructor

Constructor adalah method khusus yang secara otomatis dipanggil saat sebuah objek dibuat. Di PHP, constructor ditulis dengan nama `__construct()`. Gunanya adalah untuk memastikan objek langsung memiliki nilai yang diperlukan sejak pertama kali dibuat, sehingga tidak perlu mengisi properti satu per satu secara manual. Contoh:



```

index.php

<?php

class Students
{
    public string $nim;
    public string $name;

    // Constructor
    public function __construct(string $nim, string $name)
    {
        $this->nim = $nim;
        $this->nama = $name;
    }

    public function introduction(): string
    {
        return "Iam {$this->name}, NIM {$this->nim}";
    }
}

// Objects are filled immediately upon creation.
$std1 = new Students("202310370311238", "Alif", "Teknik Informatika");
$std2 = new Students("202310370311001", "Nisrina", "Sistem Informasi");

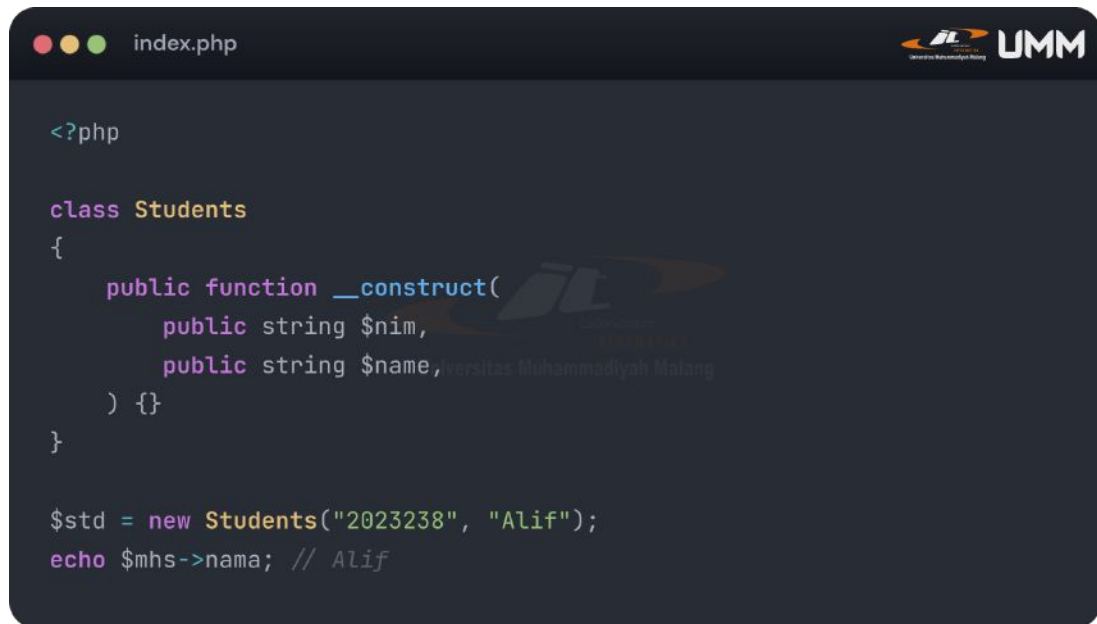
echo $std1->introduction();
// Output: I'am Alif, NIM 202310370311238

echo $std2->introduction();
// Output: I'am Nisrina, NIM 202310370311001
    
```

Gambar 2.3: Constructor PHP OOP



Selain menggunakan cara di atas, PHP versi 8 terbaru juga memperkenalkan cara yang lebih ringkas untuk mendeklarasikan constructor, yaitu Constructor Property Promotion. Contoh:



```
<?php

class Students
{
    public function __construct(
        public string $nim,
        public string $name,
    ) {}
}

$std = new Students("2023238", "Alif");
echo $std->nama; // Alif
```

Gambar 2.4: Property promotion untuk deklarasi constructor

Jika menggunakan cara tersebut, kita bisa langsung mendeklarasikan properti pada parameter constructor.

2.5. Encapsulation dan Access Modifier

Encapsulation adalah prinsip untuk menyembunyikan detail internal sebuah objek dan hanya mengekspos bagian yang perlu ditampilkan. Ini diterapkan melalui access modifier.

Tabel 2.2: Konsep encapsulation dan access modifier

Modifier	Akses
Public	Dapat diakses dari mana saja. Dari luar class, class turunan, maupun di dalam class itu sendiri
Protected	Hanya dapat diakses di dalam class itu sendiri dan class turunannya
Private	Hanya dapat diakses di dalam class itu sendiri



Mengapa konsep ini penting? Karena tidak semua data sebaiknya bisa diubah sembarangan dari luar class. Contohnya, nilai mahasiswa seharusnya tidak bisa diisi dengan angka di luar rentang yang valid, seperti **-10** atau **500**. Oleh karena itu, nilai disimpan sebagai **private** dan hanya dapat diubah melalui method **setGrade()**, yang bertugas melakukan validasi terlebih dahulu sebelum data disimpan. Contoh:



```

<?php

class Student
{
    private int $grade = 0;

    public function setGrade(int $grade): void
    {
        if ($grade >= 0 && $grade <= 100) {
            $this->grade = $grade;
        }
    }

    public function getGrade(): int
    {
        return $this->grade;
    }
}

$student = new Student();

$student->setGrade(90);

echo $student->getGrade(); // 90
    
```

Gambar 2.5: Contoh implementasi encapsulation PHP OOP

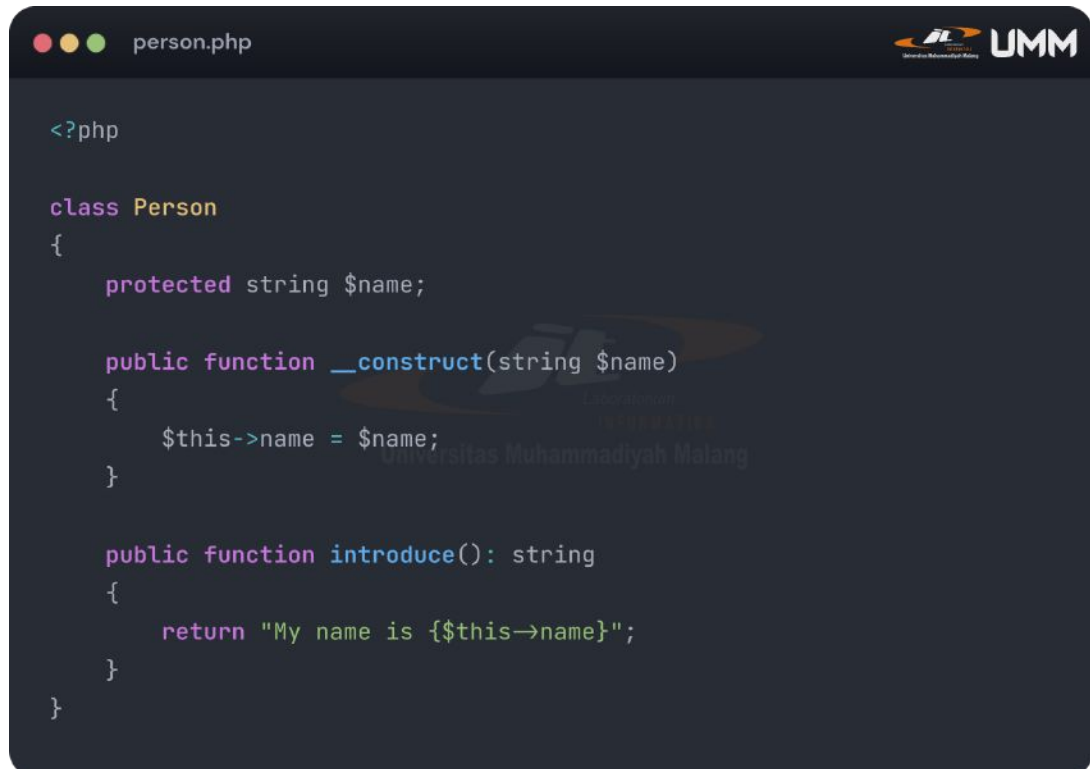
Catatan: Gunakan **private** sebagai default untuk property. Gunakan **public** hanya untuk method yang memang perlu dipanggil dari luar. Gunakan **protected** ketika property atau method perlu diakses oleh class turunan.



2.6. Inheritance

Inheritance (pewarisan) memungkinkan sebuah class mewarisi property dan method dari class lain. Class yang diwarisi disebut parent class (atau superclass), dan class yang mewarisi disebut child class (atau subclass). Konsep ini bertujuan untuk menghindari duplikasi kode ketika beberapa entitas memiliki karakteristik yang sama. Contoh:

Class person



```

person.php

<?php

class Person
{
    protected string $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function introduce(): string
    {
        return "My name is {$this->name}";
    }
}

```



Class student

```

person.php

<?php

class Person
{
    protected string $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function introduce(): string
    {
        return "My name is {$this->name}";
    }
}

```

Class teacher

```

teacher.php

<?php

class Teacher extends Person
{
    public function introduce(): string
    {
        return "I am a teacher named {$this->name}";
    }
}

```

Pada 3 contoh kode di atas, class **Student** dan **Teacher** mewarisi (inherit) properti dan method dari class **Person**. Karena mewarisi **Person**, class **Student** dan **Teacher** otomatis memiliki properti **\$name** serta constructor yang dimiliki oleh parent class-nya.



Selain mewarisi, kedua child class juga menimpa (override) method `introduce()` milik parent class dengan implementasi yang berbeda. Meskipun method yang dipanggil sama, yaitu `introduce()`, hasil yang diberikan berbeda tergantung pada jenis objeknya. Konsep ini disebut polymorphism, yaitu kemampuan objek yang berbeda untuk memberikan perilaku yang berbeda melalui method yang sama. Jika dipanggil, output yang dihasilkan akan berbeda:



```

index.php

$student = new Student("Alice");
$teacher = new Teacher("Bob");

echo $student->introduce();
// Output: I am a student named Alice

echo $teacher->introduce();
// Output: I am a teacher named Bob

```

Gambar 2.6: Contoh polymorphism PHP OOP

3. Mengorganisir kode pada PHP

3.1. Struktur Project

Pada aplikasi sederhana, seluruh kode sering kali ditulis dalam satu file saja. Contoh:

```

project/
└── index.php

```

Cara tersebut masih dapat digunakan jika skala aplikasinya masih kecil. Namun, ketika jumlah class bertambah, kode akan semakin rumit untuk dibaca. Ketika aplikasi semakin berkembang, kelas yang memiliki fungsi serupa biasanya dikelompokkan ke dalam satu folder. Contoh:

```

project/
├── Models/
│   └── Product.php
├── Services/
│   └── PaymentService.php
└── index.php

```



Struktur project yang baik akan mempermudah kita dalam maintenance, mengurangi duplikasi kode dan mempermudah kerja sama apabila bekerja dengan tim.

3.2. Namespace

Ketika aplikasi semakin besar, kemungkinan terjadinya konflik nama class juga semakin tinggi. Kegunaan namespace adalah untuk mengorganisasi kode dengan lebih rapi dan terstruktur, sehingga lebih mudah untuk di-maintenance. Cara kerja namespace mirip dengan cara kerja folder dan file. Dalam folder tidak bisa membuat 2 file dengan nama yang sama, tetapi hal tersebut bisa dilakukan jika foldernya berbeda. Contoh:

```
App/
├── Admin/
│   └── user.php
└── Services/
    └── user.php
```

Dan penulisan class user di dalam namespace bisa seperti ini:



```
<?php

namespace App\Admin;

class User
{
    public string $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function role()
    {
        return "Admin";
    }
}
```

Gambar 3.1: Contoh implementasi namespace PHP OOP



Pada contoh di atas, kedua file memiliki nama yang sama, yaitu `user.php`, tetapi tidak menimbulkan konflik karena berada di folder yang berbeda. Jika ingin menggunakan class yang berada pada namespace di file lain, gunakan kata kunci `use`. Contoh:



```
<?php

use App\Admin\User

$admin = new AdminUser("Budi");

echo $admin->role();
// output: "Admin"
```

Gambar 3.2: Penggunaan keyword use

3.3. Type Hinting dan Return Type

PHP versi 8.x mendorong penulisan kode yang lebih eksplisit dan aman melalui deklarasi tipe data. Dengan menentukan tipe data yang digunakan, PHP dapat membantu mendeteksi kesalahan lebih awal serta membuat kode lebih mudah dibaca dan dipahami oleh programmer lain saat bekerja dalam tim.

- Type Hinting digunakan untuk menentukan tipe data yang harus diterima oleh parameter suatu function atau method
- Return Type digunakan untuk menentukan tipe data yang harus dikembalikan oleh function atau method

Contohnya seperti ini:

Type Hinting



```
<?php

function greet(string $name)
{
    echo "Hello, $name";
}

greet("Alice");
```

Gambar 3.3: Type hinting PHP OOP

Pada contoh di atas, parameter `$name` harus berupa `string`. Selain itu, akan terjadi `error`.



Return Type



```

index.php

<?php

function add(int $a, int $b): int
{
    return $a + $b;
}

echo add(5, 3);
    
```

Gambar 3.4: Return type PHP OOP

Pada contoh di atas, `int` setelah parameter merupakan return type yang menandakan bahwa function harus mengembalikan nilai bertipe integer. Jika function mengembalikan tipe data yang berbeda dari yang telah dideklarasikan, program PHP akan menampilkan pesan error.

4. Perancangan backend engine sederhana

Pada bagian ini, seluruh konsep yang telah dipelajari sebelumnya akan digabungkan menjadi sebuah studi kasus sederhana. Tujuannya adalah memahami bagaimana class, object, encapsulation, inheritance, dan konsep OOP lainnya bekerja bersama untuk membangun sistem.

4.1. Identifikasi entitas sistem

Sebelum mulai menulis kode, langkah pertama dalam perancangan sistem adalah mengidentifikasi entitas yang terlibat. Dalam OOP, setiap entitas umumnya direpresentasikan sebagai sebuah class yang memiliki atribut (data) dan metode (perilaku). Dengan melakukan identifikasi entitas terlebih dahulu, kita dapat menentukan:

- Class apa saja yang perlu dibuat
- Data apa saja yang dimiliki setiap class
- Perilaku atau fungsi apa saja yang dapat dilakukan oleh setiap class
- Hubungan antar-class dalam sistem

Sebagai contoh, pada sistem perpustakaan sederhana terdapat beberapa entitas utama yang akan dijadikan class, yaitu Book, Member dan Library. Setelah entitas berhasil diidentifikasi, langkah berikutnya adalah merancang atribut dan method yang dimiliki oleh masing-masing class. Contoh:



Tabel 4: Contoh hasil identifikasi entitas sistem untuk class OOP

Class	Atribut	Method
Book	title, author, available	borrow(), returnBook()
Member	name, memberId	borrowBook()
Library	books	addBook(), listBooks()

Catatan kecil:

Tabel di atas merupakan rancangan awal sistem. Saat proses implementasi, sering kali ditemukan kebutuhan tambahan. Contohnya, akan ada penambahan method `getTitle()` dan `isAvailable()` untuk membaca data buku secara aman tanpa mengakses atribut secara langsung di class Book. Hal yang sama juga bisa terjadi di class yang lain. Penambahan method seperti ini merupakan hal yang umum dalam proses pengembangan aplikasi.

4.2. Implementasi prototype backend engine

Setelah mengidentifikasi entitas, atribut, dan method yang dibutuhkan, langkah berikutnya adalah mengimplementasikan rancangan tersebut dalam bentuk kode program. Pada aplikasi yang terus berkembang, tidak disarankan untuk menulis seluruh kode dalam satu file. Setiap class ditempatkan dalam file terpisah agar lebih mudah dibaca, dikelola, dan dikembangkan. Contoh:

```
library-system/
├── src/
│   ├── Book.php
│   ├── Member.php
│   └── Library.php
└── index.php
```

Keterangan:

`src/` digunakan untuk menyimpan seluruh class

`index.php` berfungsi sebagai entry point aplikasi yang akan menjalankan sistem



Class Book

```

book.php

<?php

class Book
{
    private string $title;
    private string $author;
    private bool $available = true;

    public function __construct(string $title, string $author)
    {
        $this->title = $title;
        $this->author = $author;
    }

    public function getTitle(): string
    {
        return $this->title;
    }

    public function isAvailable(): bool
    {
        return $this->available;
    }

    public function borrow(): void
    {
        $this->available = false;
    }

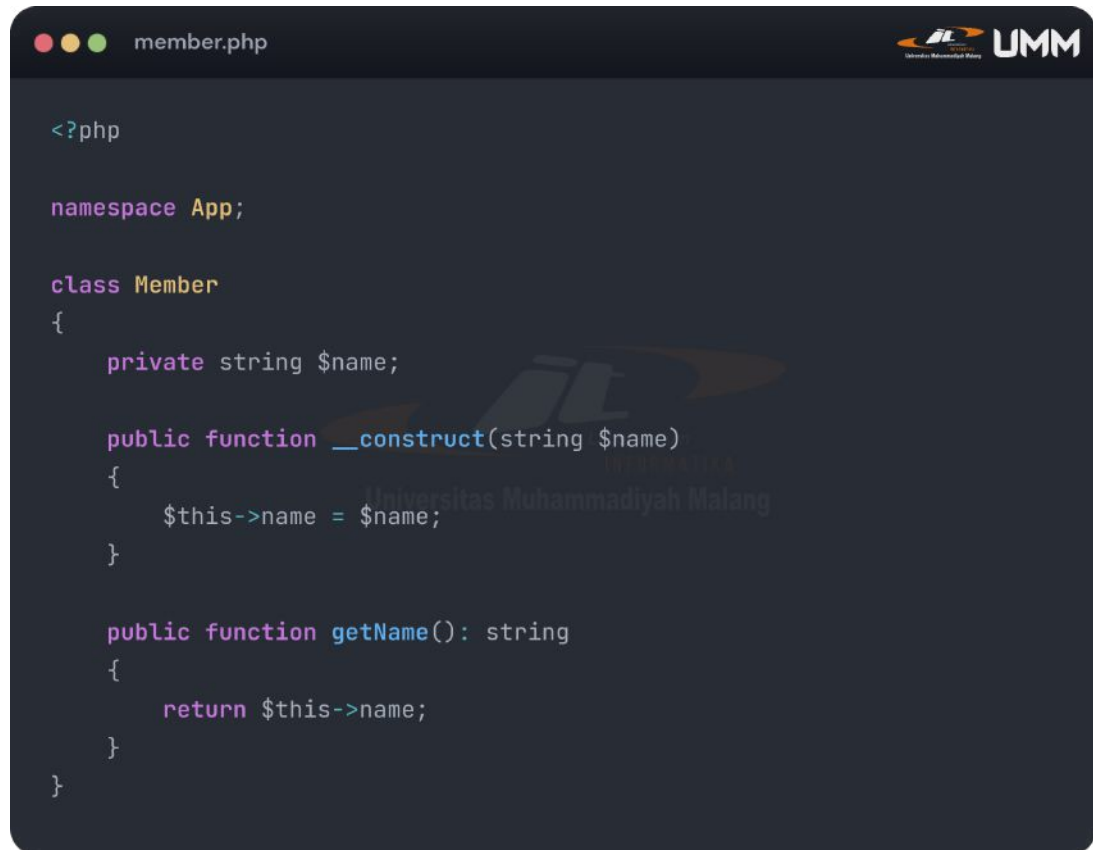
    public function returnBook(): void
    {
        $this->available = true;
    }
}

```

Gambar 4.1: Contoh class Book



Class Member



```
member.php

<?php

namespace App;

class Member
{
    private string $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function getName(): string
    {
        return $this->name;
    }
}
```

Gambar 4.2: Contoh class Member



Class Library

```
library.php

<?php

namespace App;

class Library
{
    private array $books = [];

    public function addBook(Book $book): void
    {
        $this->books[] = $book;
    }

    public function listBooks(): void
    {
        foreach ($this->books as $book) {

            $status = $book->isAvailable()
                ? "Available"
                : "Borrowed";

            echo $book->getTitle() . " - " . $status . PHP_EOL;
        }
    }
}
```

Gambar 4.3: Contoh class Library



Index (entry point)

```

index.php

<?php

require_once 'src/Book.php';
require_once 'src/Member.php';
require_once 'src/Library.php';

use App\Book;
use App\Library;
use App\Member;

$library = new Library();

$book1 = new Book(
    "Clean Code",
    "Robert C. Martin"
);

$book2 = new Book(
    "Design Patterns",
    "Gang of Four"
);

$member = new Member("Alice");

$library->addBook($book1);
$library->addBook($book2);

$book1->borrow();

echo "Member: " . $member->getName() . PHP_EOL;
echo PHP_EOL;

$library->listBooks();

```

Gambar 4.4: File index sebagai entry point



CODELAB

Mengembangkan prototype backend engine

Kembangkan prototype backend engine yang telah disiapkan di repository github berikut:

[modul3_codelab](#)

Dengan menambahkan kemampuan agar anggota perpustakaan dapat meminjam dan mengembalikan buku.

Catatan: Pada codelab ini, beberapa bagian kode telah dibuat dalam bentuk rumpang. Lengkapilah bagian yang masih rumpang tersebut sesuai petunjuk yang diberikan menggunakan konsep-konsep PHP OOP yang telah dipelajari pada modul ini.

Apabila implementasi dilakukan dengan benar, maka output program kurang lebih menjadi seperti ini:

```
Member : Alice

Library Books
Clean Code - Borrowed
Design Patterns - Available

After Returning Book
Clean Code - Available
Design Patterns - Available
```



DEMO

Pada demo praktikum ini, praktikan diminta untuk mengembangkan prototipe engine backend sederhana menggunakan bahasa pemrograman PHP dengan paradigma Object-Oriented Programming (OOP) berdasarkan studi kasus **FlipPlay**. Prototype yang dibuat tidak perlu berbentuk aplikasi web, melainkan berupa simulasi yang dijalankan melalui terminal atau command line. Fokus utama demo adalah untuk menunjukkan penerapan konsep-konsep OOP yang telah dipelajari pada modul ini.

Ketentuan Demo

Prototype yang dibuat wajib memenuhi ketentuan berikut:

1. Menggunakan minimal tiga class
2. Menerapkan constructor (`__construct`)
3. Memiliki property dan method
4. Menggunakan access modifier secara tepat
5. Menerapkan inheritance sederhana atau struktur class yang rapi
6. Menerapkan type hinting dan return type pada method yang sesuai
7. Menggunakan namespace untuk mengorganisasi class
8. Memiliki satu file utama (`index.php`) sebagai entry point untuk menjalankan simulasi

Simulasi Yang Diharapkan

Praktikan bebas menentukan alur simulasi selama masih sesuai dengan studi kasus FlipPlay. Sebagai contoh, simulasi dapat menampilkan proses:

1. Menambahkan media ke dalam library
2. Menampilkan daftar media yang tersedia
3. Mencari media berdasarkan judul
4. Menampilkan informasi detail suatu media
5. Menampilkan koleksi media milik pengguna.



RUBRIK PENILAIAN

Aspek Penilaian	Poin (%)
Codelab	Total 10%
Berhasil melengkapi kode yang kurang	5
Output sesuai dengan yang diharapkan	5
Demo	Total 80%
Simulasi sesuai studi kasus	30
Menerapkan PHP OOP	30
Implementasi sesuai dengan ketentuan	10
Program berhasil dijalankan	10
Pemahaman	Total 10%
Ketepatan menjawab pertanyaan	10



SEDIKIT CATATAN

Pada modul ini, praktikan membangun aplikasi menggunakan PHP murni untuk memahami konsep dasar Object-Oriented Programming (OOP) tanpa bantuan framework. Konsep seperti **class**, **object**, **inheritance**, **namespace**, dan **autoloading** merupakan fondasi yang juga digunakan oleh framework PHP modern, termasuk **Laravel**. Pada modul berikutnya, praktikan akan mempelajari bagaimana Laravel memanfaatkan konsep-konsep tersebut untuk menyederhanakan proses pengembangan aplikasi web yang lebih terstruktur, mudah dipelihara, dan siap dikembangkan dalam skala yang lebih besar. Semangat 😊😊

